

Skin Cancer Detection

Created time May 5, 2026 9:27 AM

CANCER BOYS

Naveen Verma (B240014AI)
Nitin Kumar Jha (B240016AI)
Kausal Sharma (B240045CS)
Pawan Kumar Limboo (B240055CS)

1. Abstract

The rapid growth of digital medical data has created strong opportunities to apply Artificial Intelligence (AI) in cancer diagnosis. This project presents a Data Warehousing and Mining (DWM) system designed to automatically collect, store, and classify skin cancer images. The system uses an Extract, Transform, Load (ETL) pipeline to gather clinical images from multiple web sources. These images are processed into a standardized format and stored in a cloud-based database using Supabase (PostgreSQL).

A Convolutional Neural Network (CNN) is integrated into the system to classify skin lesions into three main types: Melanoma (MEL), Basal Cell Carcinoma (BCC), and Squamous Cell Carcinoma (SCC). The entire system is connected through a REST API and deployed on Render, enabling real-time predictions through a web-based dashboard.

By combining automated data collection, cloud storage, and deep learning, this project provides a scalable system that supports early detection and analysis of skin cancer.

2. Introduction

2.1 Background on Skin Cancer

Skin cancer remains one of the most prevalent malignancies globally. It primarily manifests in three forms: Melanoma (MEL), which is the most aggressive and accounts for the majority of skin cancer-related fatalities; Basal Cell Carcinoma (BCC), the most common but least aggressive; and Squamous Cell Carcinoma (SCC), which carries a moderate risk of metastasis. Accurate morphological differentiation between these classes is highly complex, often requiring invasive biopsies and expert histopathological analysis.

2.2 Importance of Early Detection

The prognosis for skin cancer is heavily dependent on the stage at which it is diagnosed. If detected early, the 5-year survival rate for localized Melanoma exceeds 99%. However, this rate drops significantly if the cancer metastasizes. Consequently, accessible, rapid, and non-invasive preliminary screening tools are critical in reducing mortality rates.

2.3 Motivation for using AI and Data Warehousing

While AI algorithms (specifically CNNs) have shown dermatologist-level accuracy in classifying skin lesions, they suffer from the "data bottleneck." Training robust models requires continuous access to diverse, well-labeled, and high-quality datasets. Traditional manual dataset compilation is static and tedious. This project is motivated by the need to merge Data Warehousing methodologies with Deep Learning. By deploying an automated web-scraping ETL pipeline, the system dynamically "mines" new clinical data, standardizes it, and stores it in a centralized warehouse, thereby creating a continuously evolving repository for model training and real-time inference.

3. Literature Review

3.1 Existing Methods for Skin Cancer Detection

Traditional computational methods relied on explicit feature extraction techniques such as ABCD (Asymmetry, Border irregularity, Color variegation, Diameter) scoring, combined with Support Vector Machines (SVMs) or Random Forests. While effective, these methods require manual feature engineering and often fail to capture complex, high-dimensional pixel relationships.

3.2 Use of CNNs in Medical Imaging

With the advent of Deep Learning, Convolutional Neural Networks (CNNs) have become the gold standard for computer vision tasks in healthcare. Architectures like ResNet, VGG16, and custom sequential models excel at automatically learning hierarchical spatial features (edges, textures, and shapes) directly from raw image matrices.

3.3 Limitations of Current Systems

Current open-source skin cancer systems are largely static. They are trained on frozen datasets (e.g., HAM10000) and deployed as isolated scripts. They lack dynamic data ingestion capabilities. Furthermore, few systems bridge the gap between Big Data engineering (Data Warehousing) and Machine Learning execution within a unified, web-accessible dashboard.

4. Problem Statement

To design and develop an end-to-end web application that automates the collection (mining) of unstructured skin cancer images from clinical web sources, transforms and loads this data into a structured cloud data warehouse, and utilizes a Convolutional Neural Network to provide real-time diagnostic classification (MEL, BCC, SCC) through an interactive, medical-grade user interface.

5. Objectives of the Project

- a. **Automated Data Mining:** Develop a web scraper to dynamically extract medical images and metadata from public clinical repositories.
- b. **ETL Pipeline Implementation:** Standardize incoming unstructured image data (grayscale conversion, resolution normalization) and store it in a centralized repository.
- c. **Cloud Data Warehousing:** Utilize Supabase (PostgreSQL and Object Storage) to maintain a scalable fact_lesions schema.
- d. **Deep Learning Integration:** Train and deploy a CNN model capable of classifying skin lesions with high confidence.
- e. **API & UI Development:** Build a Flask-based RESTful API to bridge the ML backend with a responsive, glassmorphism-styled HTML/CSS/JS dashboard.

6. System Architecture

6.1 Overall Architecture Diagram Explanation

The system operates on a decoupled client-server architecture. The frontend acts as the presentation layer, capturing user inputs (URLs for scraping or images for prediction). These requests are routed via HTTP/POST to the Python Flask Backend. The backend contains two primary engines: the **ETL Engine** (BeautifulSoup/Requests) and the **Inference Engine** (TensorFlow/Keras/OpenCV). Both engines interact with the **Cloud Layer** (Supabase), which handles persistent data storage.

6.2 Data Flow

1. **Extract:** The user inputs a URL into the UI. The Flask backend uses Requests and BeautifulSoup4 to parse the DOM and extract raw image bytes.
2. **Transform:** Raw images are decoded using OpenCV, converted to grayscale to remove illumination bias, and resized to a standard 224×224 tensor (or 64×64 for memory-constrained environments).

3. **Load:** The transformed images are uploaded to the Supabase Storage bucket (skin-warehouse), and the metadata (source URL, assigned label, cloud path) is inserted into the Supabase PostgreSQL fact_lesions table.
4. **Model:** The CNN accesses the warehouse to train on the data, generating a saved .h5 model.
5. **API & UI:** For real-time detection, a user uploads an image via the UI, which hits the /api/predict endpoint. The model evaluates the image and returns a JSON response containing the diagnosis and confidence percentages.

7. Technologies Used

- **Programming Languages:** Python 3.10+, JavaScript (ES6+), HTML5, CSS3, SQL.
- **Web Framework:** Flask (Python) with Gunicorn as the WSGI HTTP server.
- **Data Mining & ETL:** BeautifulSoup4 (HTML parsing), Requests (HTTP client).
- **Machine Learning & Vision:** TensorFlow / Keras (CNN architecture), OpenCV (cv2), NumPy, Pandas.
- **Database & Storage:** Supabase (PostgreSQL relational database and Supabase Storage for bucket hosting).
- **Deployment Environment:** Render.com (Cloud Platform-as-a-Service).

8. Data Warehousing & ETL Process

8.1 Data Extraction

The system utilizes a custom web-scraping module. When a URL is provided, the HTTP headers are forged to mimic a standard browser to prevent blocking. The DOM is parsed to locate tags, evaluating src, data-src, and srcset attributes to extract the highest-resolution image available.

8.2 Data Transformation

Data consistency is paramount for CNN accuracy. Raw image bytes downloaded in memory are converted into NumPy arrays. cv2.imdecode processes the stream. Images are converted to grayscale to force the model to learn morphological patterns (shape, border irregularity) rather than relying on color variations, which vary heavily due to lighting. The images are then uniformly resized.

8.3 Data Loading & Schema Design

Processed data is securely uploaded to a Supabase bucket via the Supabase Python Client. A corresponding record is created in the PostgreSQL database. **Table:** fact_lesions

- id (Primary Key, UUID/Auto-increment)
- label (VARCHAR - e.g., 'MEL', 'BCC', 'SCC')
- source_url (TEXT - Original extraction point)
- local_path (TEXT - Supabase public URL/bucket path)
- created_at (TIMESTAMP)

9. Machine Learning Model

9.1 CNN Architecture

The core classification engine is a Sequential Convolutional Neural Network. The architecture typically consists of 3 convolutional blocks:

- **Block 1:** Conv2D (32 filters, 3×3 kernel, ReLU) → MaxPooling2D.
- **Block 2:** Conv2D (64 filters, 3×3 kernel, ReLU) → MaxPooling2D.
- **Block 3:** Conv2D (128 filters, 3×3 kernel, ReLU) → MaxPooling2D → Flatten.

- **Dense Layer:** Fully connected layer with Dropout to prevent overfitting, followed by a Softmax output layer matching the number of target classes (3).

(Note: In production environments with strict RAM limits, parameters are scaled down, and inference generation dynamically adjusts to ensure server stability).

9.2 Training & Classification

The model retrieves tensors from the warehouse, compiling them into training batches. It utilizes the Adam optimizer and Categorical Crossentropy loss function. Upon completion, the model categorizes unseen lesions into Melanoma, Basal Cell Carcinoma, or Squamous Cell Carcinoma, providing a percentage-based probability distribution for each.

10. API Design

The backend is structured as a RESTful service to ensure frontend decoupling.

- **POST /api/scrape:** Accepts JSON payload { "url": "...", "label": "MEL" }. Triggers the ETL pipeline, scrapes images, uploads to Supabase, and returns a JSON array of saved database records.
- **POST /api/predict:** Accepts multipart/form-data containing an image file. Returns JSON: { "prediction": "MEL", "confidence": 94.5, "risk": "High" }.
- **GET /api/records & /api/stats:** Connects via Supabase Client to perform SQL SELECT queries, returning aggregate counts and historical data for the dashboard's analytics charts.

11. Implementation Details

The application initialization begins in `server.py`, which loads environment variables (.env) for secure Supabase authentication. The server instantiates the Flask application and registers CORS. The ETL logic is embedded directly into the API routes to allow real-time mining triggered by user UI interaction. The frontend (index.html, app.js) is served statically but remains highly dynamic, using asynchronous fetch() API calls to continuously poll the backend for updates, rendering loaders during heavy ML tasks or network I/O.

12. Results & Output

12.1 Model Predictions

The deployed CNN successfully identifies morphological markers of skin cancer, outputting a primary diagnosis accompanied by a confidence matrix (e.g., 94% MEL, 4% BCC, 2% SCC). A risk assessment (High/Moderate) is appended to assist clinical interpretations.

12.2 Dashboard UI

The user interface achieves a premium, clinical aesthetic using CSS Glassmorphism. It features specific tabs for Overview, Detection, Scraping, and the Registry. *(Instructor Note: Insert screenshots of the 1. Dashboard Home, 2. Live Scraper Logs, and 3. Prediction Result here).*

12.3 Dataset Distribution

Through the DWM pipeline, the system tracks the origin of all data, rendering real-time statistics that show the breakdown of the warehouse by class (MEL vs. BCC) and by source domain.

13. Advantages

- **Automated Scalability:** The web-scraping ETL pipeline eliminates manual dataset collection, allowing the warehouse to grow exponentially.
- **Serverless Data Management:** Supabase handles PostgreSQL and blob storage securely in the cloud, removing the need for local hard-drive storage management.
- **Intuitive UI/UX:** The separation of the complex backend from the clean, glass-themed frontend ensures high usability for medical professionals.

14. Limitations

- **Cloud Provider Constraints:** Free-tier limits on PaaS providers like Render restrict active RAM, which can throttle the training of highly complex, deep CNN architectures.
- **Web Scraping Volatility:** Reliance on unstructured DOM elements means changes to source clinical websites can break the extraction logic.
- **Clinical Accuracy:** As an academic project, the model serves as an initial screening tool and lacks the rigorous regulatory validation required for absolute clinical diagnosis.

15. Future Scope

Future iterations of this project could integrate robust data augmentation during the ETL phase (rotations, flips) to increase model resilience. Transitioning to a microservices architecture where the ML training pipeline runs on a dedicated GPU instance (e.g., AWS EC2) while the Flask API remains on Render would solve memory bottlenecks. Finally, incorporating a federated learning approach could allow multiple clinical instances to update the model without sharing sensitive raw patient data.

16. Conclusion

The "Skin Cancer Detection using Data Warehousing and Mining" project successfully bridges the gap between Big Data Engineering and Artificial Intelligence. By developing an automated ETL pipeline that continuously feeds a Supabase-backed cloud warehouse, the project circumvents the traditional limitations of static AI datasets. Coupled with a Convolutional Neural Network and a responsive RESTful Flask API, the application provides a scalable, end-to-end ecosystem for the real-time classification of skin cancer lesions, serving as a robust proof-of-concept for next-generation medical informatics tools.

17. References

- a. Esteva, A., et al. (2017). *Dermatologist-level classification of skin cancer with deep neural networks*. Nature, 542(7639), 115-118.
- b. Supabase Documentation. (2025). *PostgreSQL and Storage API Reference*. Retrieved from <https://supabase.com/docs>
- c. Chollet, F. (2015). *Keras: Deep Learning for humans*. GitHub repository.
- d. Richardson, L. (2020). *Beautiful Soup Documentation*. Retrieved from <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>
- e. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media.

18. Appendix

Appendix A: Core ETL Pipeline Execution (Python/Supabase)

```
# Extract and Transform

response = requests.get(img_url)

img_bytes = response.content

# Load: Supabase Storage

supabase.storage.from_("skin-warehouse").upload(

path=filename,
```

```
file=img_bytes,  
  
file_options={"content-type": "image/jpeg"}  
  
)  
  
# Load: Supabase Relational Database  
  
supabase.table("fact_lesions").insert({  
  
"source_url": img_url,  
  
"label": label,  
  
"local_path": filename  
  
}).execute()
```